Gaussian Splatting for Physical Inverse Problems

MS Thesis I — Progress Report

Muhammad Hussain Qureshi [Roll no.]
Advisor: [Advisor name]
Faculty of Sciences, Information Technology University, Lahore

November 8, 2026

Abstract

We use Gaussian Splatting as a representation for physical fields that are observed only through a known forward model, using datasets and operators from InverseBench. 3D Gaussians reconstruct cells from scattered electromagnetic fields (inverse scattering), and 4D Gaussians with an MLP or Mamba deformation network reconstruct 2D Navier–Stokes flow over time from sparse observations. All three planned experiments run end to end, the code is in a structured GitHub repository, and the thesis document is drafted. This report summarises the method, the results so far (including negative ones), the experiments currently running (a true 3D scattering model, sharper kernels, more samples), and questions for the next meeting.

1 Status at a glance

Item	Status	Note
3D Gaussians on inverse scattering (2D and 3D)	done	2D cells, synthetic and real (BBBC024) 3D nuclei
4D Gaussians on Navier–Stokes	done	held-out frames, 100% and 10% observations
Mamba as the motion model	done	MLP vs. Mamba vs. bidirectional Mamba
True 3D scattering model	running	sensors on a sphere; results expected [date]
Sharper (generalised) Gaussian kernels	running	targets the BBBC024 edge problem
More samples with spread (10 cells)	running	mean ± std reported
GitHub repository (standard structure)	done	to be transferred to the thesis account
Thesis document (LaTeX, 37 pages)	draft	all required chapters
Slides and Open House poster (A0)	draft	3D panel updated after current runs

2 Method

Representation. A field f on a grid is a sum of anisotropic Gaussians,

$$f_\Theta(\boldsymbol{x}) = \sum_{i=1}^{N} a_i \exp\Big(-\tfrac{1}{2}\big[(\boldsymbol{x}-\boldsymbol{\mu}_i)^\top \Sigma_i^{-1} (\boldsymbol{x}-\boldsymbol{\mu}_i)\big]^\beta \Big), \qquad \Sigma_i = R_i S_i^2 R_i^\top, \tag{1}$$

with position $\boldsymbol{\mu}_i$, scales S_i, rotation R_i (angle in 2D, quaternion in 3D) and amplitude $a_i \geq 0$ (signed for vorticity). $\beta = 1$ is the usual Gaussian; $\beta > 1$ gives flatter tops and sharper edges. Each Gaussian is evaluated only in a local window around its centre, as in 3DGS.

Physics instead of a camera. The camera rasteriser of 3DGS is replaced by the forward operator $\mathcal{A}$ of the problem (Figure 1). Parameters are fitted with Adam to

$$\min_{\Theta} \frac{\|\mathcal{A}f_\Theta - \boldsymbol{y}\|_2^2}{\|\boldsymbol{y}\|_2^2} + \lambda\,\overline{f_\Theta}, \tag{2}$$

where the second term is a sparsity penalty on the mean intensity. New Gaussians are added where the back-projected residual $\mathrm{Re}\,\mathcal{A}^H(\boldsymbol{y} - \mathcal{A}f_\Theta)$ is largest; Gaussians with negligible amplitude are pruned. The baseline optimises every pixel or voxel directly with the same loss.

Inverse scattering. The linear (Born) model of InverseBench: $y_{t,r} = \Delta x\,\Delta y \sum_p f_p\,\overline{u^{\mathrm{inc}}_{p,t}}\,G_{p,r}$, with 20 transmitters and 360, 180 or 60 receivers on a circle (our implementation matches InverseBench to 3×10^{-7}). For 3D volumes the operator was first applied slice by slice. The new *true 3D* model (Section 4) uses 3D Green's functions with sensors on a sphere.

4D Gaussians for flow. Canonical Gaussians are moved over time by a deformation network $\Delta\boldsymbol{\mu}_i(t), \Delta s_i(t), \Delta a_i(t) = D_\phi(\boldsymbol{\mu}_i, t)$: an MLP with Fourier features (as in 4D Gaussian Splatting) or a Mamba selective state-space model that reads each Gaussian's trajectory over all time steps as a sequence (causal or bidirectional). A temporal smoothness term $\lambda_T \sum_t \|D_\phi(\cdot, t{+}1) - D_\phi(\cdot, t)\|^2$ links observed and unobserved times.

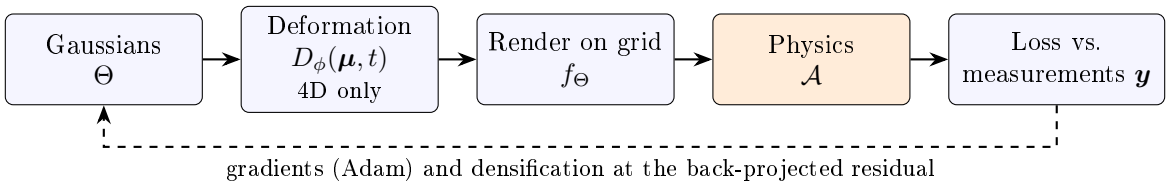


Figure 1: Pipeline: the camera of Gaussian Splatting is replaced by the physics operator.

3 Results so far

All runs: NVIDIA T4, 3000 iterations, InverseBench noise level. PSNR in dB / SSIM, means over samples.

3.1 2D inverse scattering (InverseBench cells)

Without regularisation, adding Gaussians made results *worse*: they grew ghost copies of the cell in regions the receivers cannot see (the operator's null space, Figure 2). A sparsity term removes the ghosts. With it, Gaussians beat the plain pixel grid in SSIM at 60 receivers, but a pixel grid given the same sparsity term matches or beats the Gaussians (Table 1): the gain comes from the prior, not from the representation.

Table 1: 2D inverse scattering, 5 cells (PSNR / SSIM).

Method	360 receivers	180 receivers	60 receivers
Gaussians, no sparsity (3 cells)	30.7 / 0.959	30.4 / 0.957	26.5 / 0.814
Gaussians + sparsity ($\lambda = 0.3$)	30.8 / 0.980	30.6 / 0.979	27.1 / 0.924
Pixel grid	**36.1** / 0.984	**34.7** / 0.974	**28.6** / 0.886
Pixel grid + sparsity ($\lambda = 0.3$)	33.2 / **0.985**	32.5 / **0.983**	27.7 / **0.933**

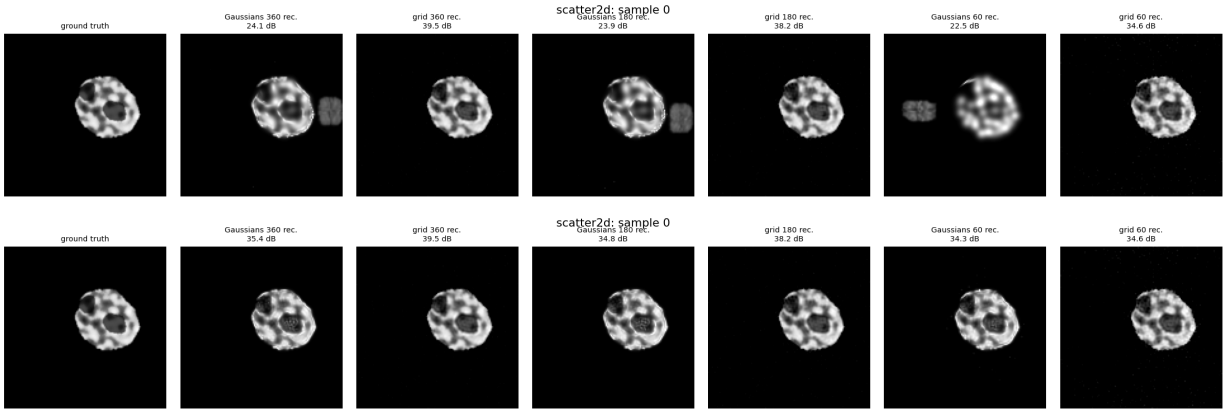


Figure 2: Cell 0. Top: aggressive densification, no sparsity: a ghost copy appears beside the cell. Bottom: with sparsity the ghost is gone. Columns: truth, then Gaussians and pixel grid at 360, 180 and 60 receivers.

3.2 3D inverse scattering (slice-wise operator)

On smooth synthetic nuclei, Gaussians barely change as receivers drop (40.8 $\rightarrow$ 40.9 dB) while the voxel grid falls to 37.3 dB and becomes speckled. On real BBBC024 nuclei, which are binary masks with sharp boundaries, Gaussians stay near 27–28 dB while voxels reach 35–42 dB: smooth kernels soften the edges (Table 2, Figure 3).

Table 2: 3D inverse scattering, $32 \times 128 \times 128$ volumes, 3 volumes (PSNR / SSIM).

Volumes	Method	360 receivers	180 receivers	60 receivers
Synthetic	Gaussians	40.8 / **0.994**	40.6 / **0.994**	**40.9** / **0.988**
	Voxel grid	**44.1** / 0.987	**41.5** / 0.977	37.3 / 0.932
BBBC024	Gaussians	27.9 / 0.967	27.6 / 0.965	27.0 / 0.943
	Voxel grid	**41.8** / 0.989	**40.3** / 0.980	**35.3** / 0.936
	Gaussians + sparsity	28.4 / 0.974	28.4 / 0.974	27.4 / 0.955
	Voxel grid + sparsity	38.5 / **0.996**	38.0 / **0.995**	34.4 / **0.971**

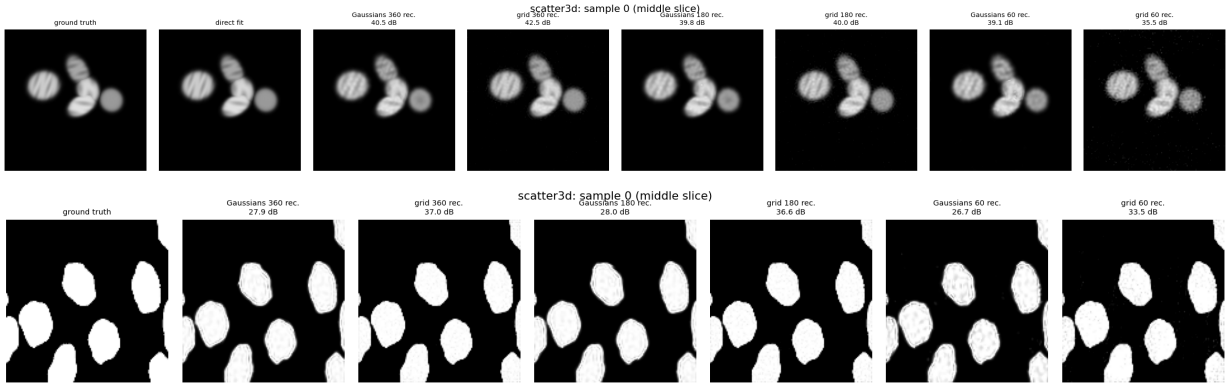


Figure 3: Middle slices. Top: synthetic volume (truth, direct fit, then Gaussians and voxels at 360/180/60 receivers); voxels become speckled at 60 receivers. Bottom: BBBC024 volume; Gaussians soften the sharp boundaries.

3.3 4D Navier–Stokes

Static Gaussians reach only 17 dB; a deformation network adds about 10 dB. Without regularisation, Mamba fits observed frames much better than the MLP but does not generalise better to unseen times. With temporal smoothness, Mamba beats the MLP by 2.4 dB on unseen frames when only 10% of pixels are observed, on each of the three sequences (Table 3, Figure 4). The bidirectional variant gives no further gain and is twice as slow.

Table 3: 4D Navier–Stokes, PSNR (dB) on held-out frames; observed-frame PSNR in brackets; 3 sequences.

Observed pixels	Static	MLP	Mamba	Bidirectional Mamba
100%	17.0 (16.8)	27.6 (30.3)	**27.9** (36.1)	–
100% + smoothness	–	27.4 (30.6)	27.2 (35.2)	26.4 (36.5)
10%	16.0 (15.9)	23.2 (24.0)	23.1 (27.7)	–
10% + smoothness	–	23.0 (24.1)	**25.4** (28.9)	25.1 (29.6)
Time per sequence	3 min	3 min	15–28 min	52 min

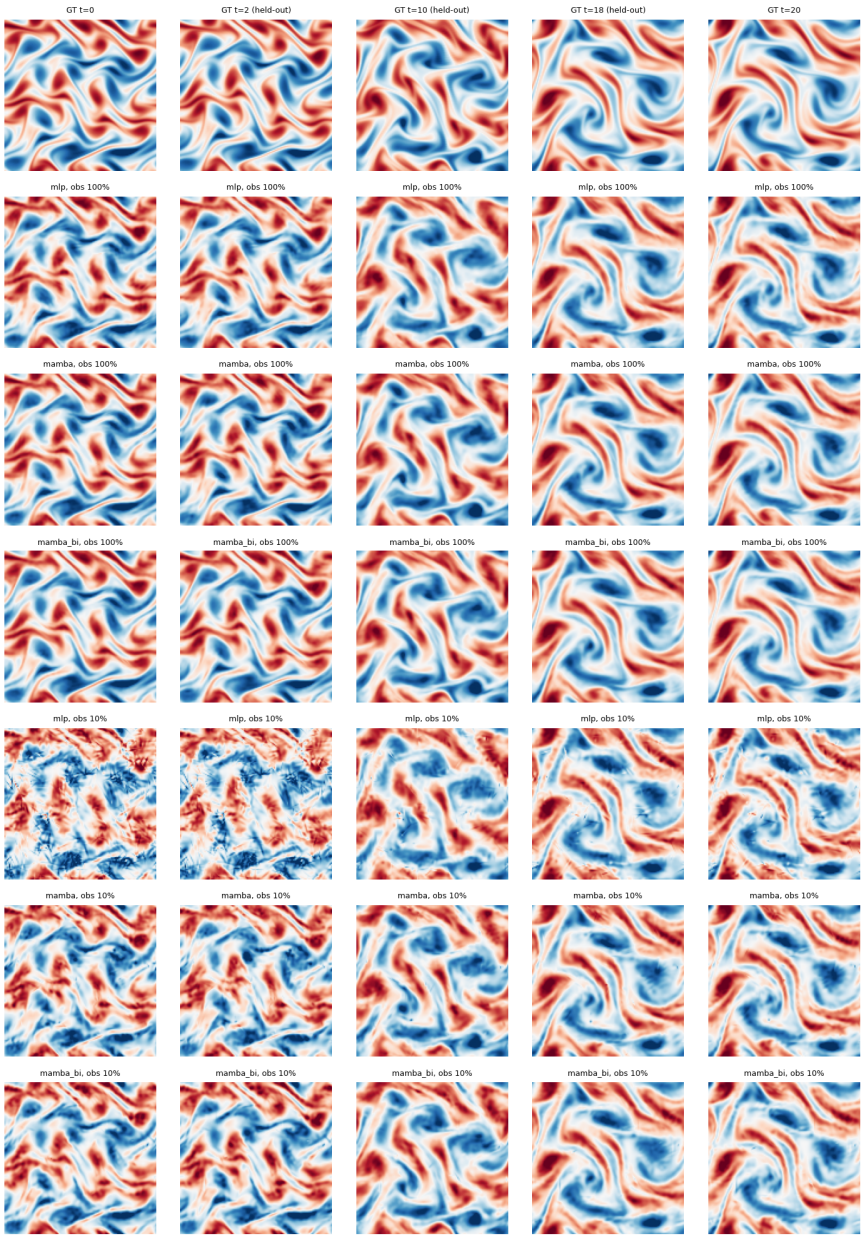


Figure 4: Vorticity of sequence 0 at $t = 0, 2^*, 10^*, 18^*, 20$ (* never observed). Rows: truth; MLP, Mamba and bidirectional Mamba with 100% and then 10% of pixels observed. With 10% the MLP shows streaks; Mamba stays coherent.

4 Currently running

True 3D scattering. Following your request, the slice-wise operator is replaced by a first-order Born model in 3D,

$$y_{t,r} = k^2 \, \Delta V \sum_p f_p \, G(\|\boldsymbol{x}_t - \boldsymbol{x}_p\|) \, G(\|\boldsymbol{x}_r - \boldsymbol{x}_p\|), \qquad G(d) = \frac{e^{\mathrm{i}kd}}{4\pi d}, \tag{3}$$

with the same voxel size, wavelength and sensor radius as InverseBench, 60 transmitters and 360/180/60 receivers spread evenly over a sphere around the volume, and noise at the SNR of the 2D benchmark. The implementation matches the analytic response of a point scatterer to 10^{-7} and passes an adjoint test. Runs on BBBC024 and synthetic nuclei, with Gaussians and a voxel grid: [results pending].

Sharper kernels and more samples. Generalised Gaussians with $\beta = 4$ on BBBC024 (both operators), and 2D runs on 10 cells with mean $\pm$ standard deviation. In a small test on binary masks, $\beta = 4$ improved a direct fit by about 3 dB. [results pending].

5 Code and documents

- Repository: `https://github.com/muhammadhussainqureshi/thesis-gs` (`src/`, `bin/`, `configs/`, `notebooks/`, `results/`, `tests/`, `docs/`). Every run stores its exact settings, per-sample metrics and figures. To be transferred to the thesis account with the required collaborators.
- Thesis document: LaTeX in `docs/thesis/` (background, literature review, problem and scope, methodology, formulation, setup, results, discussion and Thesis II plan).

6 Questions for the meeting

1. Is the true 3D model above what you had in mind, and is it sufficient for Thesis I?
2. Is reconstructing the flow from sparse observations acceptable for the 4D task, or should we add a physics term (the Navier–Stokes residual) or recover the initial condition through the solver, as InverseBench does?
3. Should the next effort go to that physics-informed 4D model, or to extending the literature review?
4. What is the exact list of Thesis I targets, and is there a required thesis template?